

1. SQL Data Definition Language (DDL)

```
CREATE DATABASE Practical01;  
USE Practical01;
```

```
CREATE TABLE Departments (  
    DeptID INT PRIMARY KEY,  
    DeptName VARCHAR(50) NOT NULL UNIQUE  
);
```

```
CREATE TABLE Employees (  
    EmpID INT PRIMARY KEY,  
    EmpName VARCHAR(50) NOT NULL,  
    Salary DECIMAL(10,2) CHECK (Salary > 0),  
    DeptID INT,  
    FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)  
);
```

```
ALTER TABLE Employees ADD HireDate DATE;  
RENAME TABLE Employees TO Staff;  
TRUNCATE TABLE Staff;  
DROP TABLE Staff;  
DROP TABLE Departments;
```

2. DML with Arithmetic Operators

```
CREATE DATABASE Practical02;  
USE Practical02;
```

```
CREATE TABLE Employees (  
    EmpID INT PRIMARY KEY,  
    EmpName VARCHAR(50),  
    Salary DECIMAL(10,2)  
);
```

```
INSERT INTO Employees (EmpID, EmpName, Salary) VALUES (1, 'Alice', 50000);  
INSERT INTO Employees (EmpID, EmpName, Salary) VALUES (2, 'Bob', 35000);
```

```
UPDATE Employees SET Salary = Salary + 5000 WHERE EmpID = 1;
```

```
DELETE FROM Employees WHERE Salary < 40000;
```

3. DML with Logical Operators

```
CREATE DATABASE Practical03;
```

```
USE Practical03;
```

```
CREATE TABLE Employees (  
    EmpID INT PRIMARY KEY,  
    EmpName VARCHAR(50),  
    Salary DECIMAL(10,2),  
    DeptID INT  
);
```

```
INSERT INTO Employees (EmpID, EmpName, Salary, DeptID) VALUES (1, 'Alice', 50000,  
101);
```

```
INSERT INTO Employees (EmpID, EmpName, Salary, DeptID) VALUES (2, 'Bob', 60000,  
102);
```

```
INSERT INTO Employees (EmpID, EmpName, Salary, DeptID) VALUES (3, 'Charlie', NULL,  
102);
```

```
UPDATE Employees SET Salary = 65000 WHERE DeptID = 102 AND Salary < 65000;
```

```
DELETE FROM Employees WHERE DeptID = 101 OR Salary IS NULL;
```

4. DML with Set Operators

```
CREATE DATABASE Practical04;
```

```
USE Practical04;
```

```
CREATE TABLE Employees (  

```

```

    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    Salary DECIMAL(10,2),
    DeptID INT
);

INSERT INTO Employees VALUES (1, 'Alice', 45000, 101);
INSERT INTO Employees VALUES (2, 'Bob', 60000, 102);
INSERT INTO Employees VALUES (3, 'Charlie', 55000, 101);

SELECT EmpName FROM Employees WHERE DeptID = 101
UNION
SELECT EmpName FROM Employees WHERE Salary > 50000;

```

5. Pattern Matching and String Functions

```

CREATE DATABASE Practical05;
USE Practical05;

CREATE TABLE Employees (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    DeptID INT
);

INSERT INTO Employees VALUES (1, 'Alice', 101);
INSERT INTO Employees VALUES (2, 'Adam', 102);
INSERT INTO Employees VALUES (3, 'Bob', 101);

SELECT UPPER(EmpName) FROM Employees WHERE EmpName LIKE 'A%';

SELECT CONCAT(EmpName, ' works in department ', DeptID) AS EmployeeInfo FROM
Employees;

```

6. Access Control (DCL)

```
CREATE DATABASE Practical06;
USE Practical06;

CREATE TABLE Employees (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50)
);

CREATE ROLE 'data_entry_role';

GRANT SELECT, INSERT ON Employees TO 'data_entry_role';

REVOKE INSERT ON Employees FROM 'data_entry_role';
```

7. Transaction Control (TCL)

```
CREATE DATABASE Practical07;
USE Practical07;

CREATE TABLE Employees (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50)
);

START TRANSACTION;

INSERT INTO Employees (EmpID, EmpName) VALUES (1, 'Alice');
INSERT INTO Employees (EmpID, EmpName) VALUES (2, 'Bob');

SAVEPOINT backup_point;

DELETE FROM Employees WHERE EmpID = 2;

ROLLBACK TO backup_point;

COMMIT;
```

8. Aggregate Functions

```
CREATE DATABASE Practical08;  
USE Practical08;
```

```
CREATE TABLE Employees (  
    EmpID INT PRIMARY KEY,  
    DeptID INT,  
    Salary DECIMAL(10,2)  
);
```

```
INSERT INTO Employees VALUES (1, 101, 60000);  
INSERT INTO Employees VALUES (2, 101, 50000);  
INSERT INTO Employees VALUES (3, 102, 40000);
```

```
SELECT DeptID, SUM(Salary) AS TotalSalary  
FROM Employees  
GROUP BY DeptID  
HAVING SUM(Salary) > 100000;
```

9. JOIN Operations and Views

```
CREATE DATABASE Practical09;  
USE Practical09;
```

```
CREATE TABLE Departments (  
    DeptID INT PRIMARY KEY,  
    DeptName VARCHAR(50)  
);
```

```
CREATE TABLE Employees (  
    EmpID INT PRIMARY KEY,  
    EmpName VARCHAR(50),  
    DeptID INT
```

```
);
```

```
INSERT INTO Departments VALUES (101, 'HR'), (102, 'IT');  
INSERT INTO Employees VALUES (1, 'Alice', 101), (2, 'Bob', 102);
```

```
CREATE VIEW EmployeeDeptView AS  
SELECT e.EmpName, d.DeptName  
FROM Employees e  
INNER JOIN Departments d ON e.DeptID = d.DeptID;  
  
SELECT * FROM EmployeeDeptView;
```

10. Subqueries

```
CREATE DATABASE Practical10;  
USE Practical10;
```

```
CREATE TABLE Departments (  
    DeptID INT PRIMARY KEY,  
    DeptName VARCHAR(50)  
);
```

```
CREATE TABLE Employees (  
    EmpID INT PRIMARY KEY,  
    EmpName VARCHAR(50),  
    DeptID INT  
);
```

```
INSERT INTO Departments VALUES (101, 'HR'), (102, 'IT');  
INSERT INTO Employees VALUES (1, 'Alice', 101), (2, 'Bob', 102);
```

```
SELECT EmpName  
FROM Employees  
WHERE DeptID IN (SELECT DeptID FROM Departments WHERE DeptName = 'IT');
```

11. Stored Procedures with Cursors

```
CREATE DATABASE Practical11;  
USE Practical11;
```

```
CREATE TABLE Employees (  
    EmpID INT PRIMARY KEY,  
    Salary DECIMAL(10,2)  
);
```

```
INSERT INTO Employees VALUES (1, 50000), (2, 60000);
```

```
DELIMITER //
```

```
CREATE PROCEDURE UpdateSalaries()  
BEGIN  
    DECLARE done INT DEFAULT FALSE;  
    DECLARE current_emp INT;  
    DECLARE emp_cursor CURSOR FOR SELECT EmpID FROM Employees;  
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;  
  
    OPEN emp_cursor;  
  
    bonus_loop: LOOP  
        FETCH emp_cursor INTO current_emp;  
        IF done THEN  
            LEAVE bonus_loop;  
        END IF;  
  
        UPDATE Employees SET Salary = Salary * 1.10 WHERE EmpID = current_emp;  
    END LOOP;  
  
    CLOSE emp_cursor;  
END //
```

```
DELIMITER ;  
  
CALL UpdateSalaries();
```

12. Database Triggers

```
CREATE DATABASE Practical12;
USE Practical12;

CREATE TABLE Employees (
    EmpID INT PRIMARY KEY,
    Salary DECIMAL(10,2)
);

CREATE TABLE AuditLog (
    LogID INT AUTO_INCREMENT PRIMARY KEY,
    DeletedEmpID INT
);

DELIMITER //

CREATE TRIGGER PreventNegativeSalary
BEFORE INSERT ON Employees
FOR EACH ROW
BEGIN
    IF NEW.Salary < 0 THEN
        SET NEW.Salary = 0;
    END IF;
END //

CREATE TRIGGER AuditDeletion
AFTER DELETE ON Employees
FOR EACH ROW
BEGIN
    INSERT INTO AuditLog (DeletedEmpID) VALUES (OLD.EmpID);
END //

DELIMITER ;

INSERT INTO Employees VALUES (1, -5000);

DELETE FROM Employees WHERE EmpID = 1;
```


13. CRUD Operations using MongoDB

(Note: Do not run this in a SQL compiler. This must be run in a MongoDB shell like mongosh)

use Practical13

```
db.employees.save({ _id: 1, name: "Alice", salary: 50000, dept: "IT" });
db.employees.save({ _id: 2, name: "Bob", salary: 30000, dept: "HR" });
```

```
db.employees.find({ salary: { $gt: 40000, $lt: 60000 } });
```

```
db.employees.updateOne({ _id: 1 }, { $set: { salary: 55000 } });
```

```
db.employees.deleteOne({ _id: 2 });
```

14. Aggregation and Indexing in MongoDB

(Note: Do not run this in a SQL compiler. This must be run in a MongoDB shell like mongosh)

use Practical14

```
db.employees.insertMany([
  { _id: 1, name: "Alice", salary: 50000, dept: "IT" },
  { _id: 2, name: "Bob", salary: 60000, dept: "IT" },
  { _id: 3, name: "Charlie", salary: 40000, dept: "HR" }
]);
```

```
db.employees.createIndex({ salary: 1 });
```

```
db.employees.aggregate([
  { $match: { dept: "IT" } },
```

```
{ $group: {  
  _id: "$dept",  
  totalSalary: { $sum: "$salary" }  
}}  
]);
```